
Patron de Diseno: Builder

Patron creacional para la construccion de objetos complejos paso a paso

Problema a Resolver

Cuando un objeto requiere muchos parametros, algunos opcionales, el constructor directo produce codigo dificil de leer, mantener y extender. Se identifican tres problemas concretos:

1. Constructores telescopicos

Crear multiples versiones del constructor para cubrir combinaciones opcionales genera sobrecargas que crecen exponencialmente con cada nuevo atributo.

2. Parametros sin contexto

`new Computer("i9","32GB",null,null,"Wi n11")` es valido sintacticamente, pero no comunica que representa cada argumento. Los errores al reordenar son silenciosos.

3. Estado incompleto del objeto

Con setters, el objeto existe en estado parcial entre la primera y la ultima linea de construccion. Cualquier uso intermedio puede producir errores.

Contexto del Ejemplo

Para ilustrar el patron se construye un objeto Computer con cinco atributos, tres obligatorios y dos opcionales, que requiere distintas configuraciones segun el tipo de uso:

```
public class Computer {  
    private String cpu;          // Procesador      —  
obligatorio  
    private String ram;         // Memoria RAM      —  
obligatorio  
    private String storage;     // Almacenamiento  —  
obligatorio  
    private String gpu;         // Tarjeta grafica —  
opcional  
    private String os;          // Sistema oper. —  
opcional  
}
```

Atributos opcionales

gpu y os pueden no estar presentes. Un constructor fijo obliga a pasar null explicitamente por cada campo ausente.

Multiples configuraciones

Se necesita: PC gaming, PC de oficina, servidor. Cada una con valores distintos para los mismos atributos.

Extensibilidad

Agregar un atributo nuevo rompe el constructor y todas las llamadas existentes en el sistema.

Este objeto es candidato directo para Builder: atributos opcionales, distintas variantes de configuracion y necesidad de extensibilidad sin romper codigo existente.

Que es el Patron Builder

Definicion formal

"Separar la construccion de un objeto complejo de su representacion, de tal forma que el mismo proceso de construccion pueda crear distintas representaciones."

Que es

Un patron creacional que delega la logica de construccion de un objeto a una clase especializada (Builder), separandola del objeto resultante y del codigo cliente.

Para que sirve

Permite construir objetos paso a paso, habilitando distintas configuraciones del mismo tipo de objeto sin modificar el codigo cliente ni el Director.

Como funciona

Un Director define el orden de construccion. Un ConcreteBuilder ejecuta cada paso. El objeto final solo se entrega cuando la construccion esta completa.

Estructura del Patron

Builder (interfaz)

Define el contrato: un metodo de construccion por cada parte del Product.

ConcreteBuilder

Implementa los pasos. Mantiene el estado parcial del objeto en construccion.

Director

Controla el orden de los pasos. Trabaja solo con la interfaz Builder.

Product

El objeto complejo resultante. Solo existe completo al invocar build().

Cliente → Director → Builder (interfaz) → ConcreteBuilder → Product

Componentes del Patron — Builder y ConcreteBuilder

ComputerBuilder.java

Builder — interfaz

Define el contrato. Declara un metodo por cada parte del Product. El Director trabaja unicamente con esta interfaz, nunca con la implementacion concreta.

```
public interface ComputerBuilder {

    ComputerBuilder setCPU(String cpu);
    ComputerBuilder setRAM(String ram);
    ComputerBuilder setStorage(String storage);
    ComputerBuilder setGPU(String gpu);
    ComputerBuilder setOS(String os);

    // Retorna el Product completamente
    // construido. Solo se llama al final.
    Computer build();
}
```

GamingComputerBuilder.java

ConcreteBuilder — implementacion

Implementa la interfaz. Mantiene el objeto en construccion. Cada metodo asigna el valor al objeto interno y retorna this para habilitar el encadenamiento.

```
public class GamingComputerBuilder
    implements ComputerBuilder {

    private Computer computer = new Computer();

    public ComputerBuilder setCPU(String v)
        { computer.cpu = v;      return this; }
    public ComputerBuilder setRAM(String v)
        { computer.ram = v;     return this; }
    public ComputerBuilder setStorage(String v)
        { computer.storage = v; return this; }
    public ComputerBuilder setGPU(String v)
        { computer.gpu = v;     return this; }
    public ComputerBuilder setOS(String v)
        { computer.os = v;      return this; }

    public Computer build() { return computer; }
}
```

Componentes del Patron — Director y Product

ComputerDirector.java

Director — controla el orden

Recibe el Builder por constructor. Define el orden en que se llaman los pasos. No conoce la implementacion concreta: trabaja solo con la interfaz Builder.

```
public class ComputerDirector {
    private final ComputerBuilder builder;

    public ComputerDirector(ComputerBuilder b) {
        this.builder = b;
    }

    public Computer buildGamingPC() {
        return builder
            .setCPU("Intel Core i9-14900K")
            .setRAM("32GB DDR5")
            .setStorage("1TB NVMe SSD")
            .setGPU("RTX 4090")
            .setOS("Windows 11")
            .build();
    }
    // buildOfficePC() — misma estructura,
    // distintos valores
}
```

Computer.java

Product — objeto resultante

El objeto complejo que se construye. No tiene logica de construccion propia. Solo contiene los atributos configurados por el ConcreteBuilder durante el proceso.

```
public class Computer {

    String cpu;
    String ram;
    String storage;
    String gpu;    // opcional
    String os;     // opcional

    @Override
    public String toString() {
        return "Computer { "
            + "cpu=" + cpu
            + ", ram=" + ram
            + ", gpu=" + gpu
            + ", os=" + os + " }";
    }
}
```

Flujo de Funcionamiento

1

El cliente instancia un ConcreteBuilder

Crea new GamingComputerBuilder(). El cliente elige la variante de construccion.

2

El cliente crea el Director

Pasa el ConcreteBuilder al constructor del Director. El Director solo conoce la interfaz.

3

El cliente invoca un metodo del Director

Llama buildGamingPC(). No sabe que pasos se ejecutaran internamente.

4

El Director llama los metodos del Builder

Invoca setCPU(), setRAM(), setStorage(), etc. en el orden que determina la configuracion.

5

El Builder configura el objeto interno

Cada llamada asigna un valor al atributo correspondiente del objeto en construccion y retorna this.

6

El Director invoca build()

Una vez completados todos los pasos, el Director llama build() para finalizar la construccion.

7

El ConcreteBuilder retorna el Product

Retorna el objeto completo. A partir de aqui el objeto existe con todos sus atributos configurados.

8

El cliente recibe el Product

Obtiene el objeto listo para usar. No tuvo acceso al objeto durante su construccion.

Ventajas del Patron Builder

1

Separacion de responsabilidades

La logica de como se construye el objeto queda en el Builder. El cliente no necesita conocerla. El Director no conoce la implementacion concreta.

2

Construccion controlada y completa

El objeto solo existe despues de invocar build(). No hay estado incompleto accesible. Elimina los errores de uso prematuro del objeto.

3

Flexibilidad para distintas variantes

El mismo Director produce distintos objetos recibiendo un ConcreteBuilder diferente. Agregar una nueva variante requiere solo un nuevo ConcreteBuilder.

4

Legibilidad del codigo cliente

`builder.setCPU("i9").setRAM("32GB").setOS("Linux").build()` comunica explicitamente que se configura. No requiere conocer la firma del constructor.

5

Extensibilidad sin ruptura

Agregar un atributo opcional al Product requiere un nuevo metodo en la interfaz y en los ConcreteBuilders. El cliente y el Director no se ven afectados.

Desventajas del Patron Builder

1

Incremento en el numero de clases

El patron requiere al menos cuatro clases: Product, interfaz Builder, ConcreteBuilder y Director. Para objetos simples, este overhead no esta justificado.

2

Sincronizacion entre interfaz e implementaciones

Cada vez que el Product gana un nuevo atributo obligatorio, la interfaz Builder debe actualizarse y todos los ConcreteBuilders existentes deben implementarlo.

3

El Director puede volverse un punto de rigidez

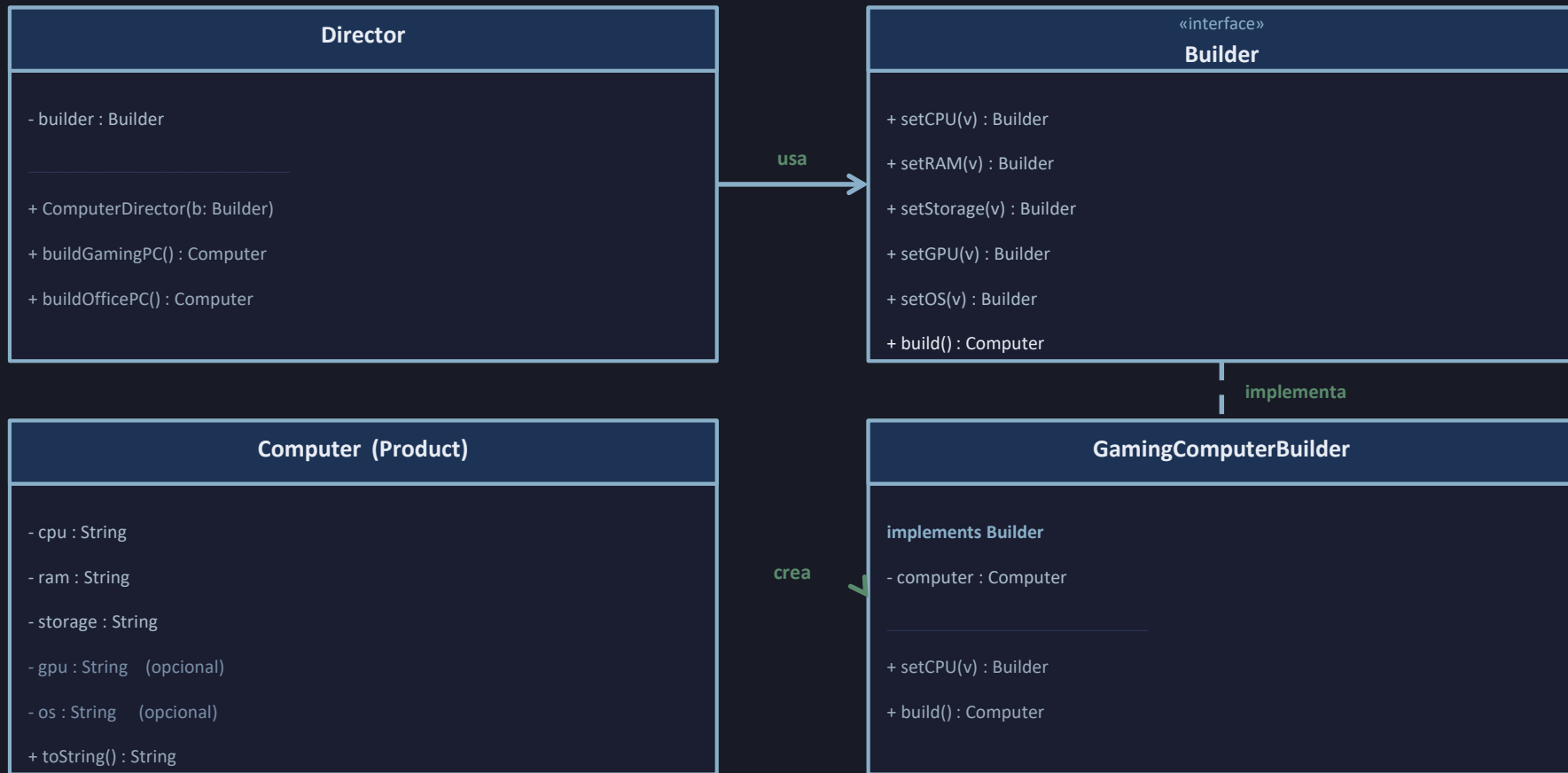
Si el Director define configuraciones muy especificas y no admite variaciones parametricas, agregar variantes con pequenas diferencias puede requerir duplicar logica.

4

Costo inicial de diseno

Implementar el patron desde cero requiere mas codigo que un constructor directo. Para objetos con dos o tres atributos, ese costo supera el beneficio obtenido.

Diagrama de Clases — Patron Builder



Usos Conocidos

1

StringBuilder (Java SE)

La clase `java.lang.StringBuilder` implementa el patron: cada metodo (`append`, `insert`) retorna `this`, y `toString()` actua como `build()`, entregando la cadena resultante solo al final.

2

Lombok @Builder (Java)

La anotacion `@Builder` de Project Lombok genera automaticamente el `ConcreteBuilder`, la interfaz y el metodo `build()` para cualquier clase Java, eliminando el codigo repetitivo de construccion.

3

OkHttp — Request.Builder (Android/Kotlin)

La libreria `OkHttp` usa `Request.Builder` para construir peticiones HTTP paso a paso: `url()`, `method()`, `header()` y `addHeader()` retornan `this`; `build()` genera el objeto `Request` inmutable.

4

DocumentBuilder (Java XML / JAXP)

`javax.xml.parsers.DocumentBuilderFactory.newDocumentBuilder()` retorna un builder para construir arboles DOM. Cada configuracion se aplica al builder antes de parsear el documento XML.

5

Spring BeanDefinitionBuilder

El framework `Spring` usa `BeanDefinitionBuilder` para definir beans de forma programatica: `addPropertyValue()`, `addConstructorArgValue()` y `getBeanDefinition()` siguen el mismo patron de construccion diferida.

Patrones Relacionados

1

Abstract Factory

Ambos patrones crean objetos complejos, pero Abstract Factory esta disenado para familias de productos relacionados. Builder se enfoca en la construccion paso a paso de un unico objeto complejo con representaciones distintas.

2

Factory Method

Builder puede usar Factory Methods internamente para crear las partes del Product. La diferencia clave: Factory Method delega la creacion a subclases en un solo paso; Builder la divide en multiples pasos coordinados.

3

Prototype

Prototype clona objetos existentes. Puede combinarse con Builder: el Director puede configurar un Prototype como plantilla y el ConcreteBuilder lo clona en build() antes de personalizarlo, reduciendo inicializacion repetitiva.

4

Composite

Los objetos Product del patron Builder suelen ser estructuras Composite (arbol de objetos). Builder simplifica la construccion de estos arboles, aislando al cliente de la logica de ensamblado recursivo.

Conclusion

1

El patron Builder separa la construccion de un objeto complejo de su representacion, encapsulando la logica de ensamblado en un ConcreteBuilder bajo el control de un Director.

2

Su principal aporte es garantizar que el objeto final solo exista cuando esta completamente construido, y que el mismo proceso de construccion pueda producir distintas representaciones al cambiar el ConcreteBuilder.

3

El patron justifica su costo cuando el objeto tiene multiples atributos opcionales, distintas variantes de configuracion, o una logica de construccion suficientemente compleja. No debe aplicarse a objetos con pocos atributos simples.

Referencias

1

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994)

Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. (GoF) — Cap. 3: Builder, pp. 97–106.

2

Freeman, E., & Robson, E. (2020)

Head First Design Patterns, 2nd ed. O'Reilly Media. — Capitulo sobre patrones creacionales: analisis practico del patron Builder con ejemplos en Java.

3

Refactoring.Guru — Builder Pattern

<https://refactoring.guru/design-patterns/builder> — Descripcion estructural, pseudocodigo, variantes (sin Director) y ejemplos en Java, C++ y Python. Consultado en 2025.

4

Oracle Java SE Documentation

<https://docs.oracle.com/en/java/api/java.base/java/lang/StringBuilder.html> — Documentacion oficial de StringBuilder como implementacion del patron en la plataforma Java.